

Язык программирования C++

Лекция 9: ссылки, перегрузка функций, new/delete, ООП, классы, инкапсуляция

Никита Лисица

2026

- **1979–1985:** Бьёрн Страуструп, работая в Bell Labs, проектирует и реализует новый язык на основе C

- **1979–1985:** Бьёрн Страуструп, работая в Bell Labs, проектирует и реализует новый язык на основе C
- **Конец 80-х – 90-е:** Язык набирает популярность

- **1979–1985:** Бьёрн Страуструп, работая в Bell Labs, проектирует и реализует новый язык на основе C
- **Конец 80-х – 90-е:** Язык набирает популярность
- **1998:** Первый официальный стандарт – C++98

- **1979–1985:** Бьёрн Страуструп, работая в Bell Labs, проектирует и реализует новый язык на основе C
- **Конец 80-х – 90-е:** Язык набирает популярность
- **1998:** Первый официальный стандарт – C++98
- **2003:** Дополнение к стандарту – C++03

- **1979–1985:** Бьёрн Страуструп, работая в Bell Labs, проектирует и реализует новый язык на основе C
- **Конец 80-х – 90-е:** Язык набирает популярность
- **1998:** Первый официальный стандарт – C++98
- **2003:** Дополнение к стандарту – C++03
- **2000-е:** Язык развивается медленно в сравнении с другими языками того же времени (Java, C#, Python, Scala, F#, ...)

- **1979–1985:** Бьёрн Страуструп, работая в Bell Labs, проектирует и реализует новый язык на основе C
- **Конец 80-х – 90-е:** Язык набирает популярность
- **1998:** Первый официальный стандарт – C++98
- **2003:** Дополнение к стандарту – C++03
- **2000-е:** Язык развивается медленно в сравнении с другими языками того же времени (Java, C#, Python, Scala, F#, ...)
- **2011:** Стандарт C++11, привнёсший много нового и изменивший взгляд на C++ (т.н. *Modern C++*)

- **1979–1985:** Бьёрн Страуструп, работая в Bell Labs, проектирует и реализует новый язык на основе C
- **Конец 80-х – 90-е:** Язык набирает популярность
- **1998:** Первый официальный стандарт – C++98
- **2003:** Дополнение к стандарту – C++03
- **2000-е:** Язык развивается медленно в сравнении с другими языками того же времени (Java, C#, Python, Scala, F#, ...)
- **2011:** Стандарт C++11, привнёсший много нового и изменивший взгляд на C++ (т.н. *Modern C++*)
- **Дальше:** Новый стандарт каждые 3 года (C++14, C++17, C++21, C++23, C++26)

- Начиная с 90х годов, C++ – **основной язык** для низкоуровневых, высокопроизводительных систем

- Начиная с 90х годов, C++ – **основной язык** для низкоуровневых, высокопроизводительных систем
- Все браузеры, программы для обработки графики и видео, крупные игровые движки и т.д. написаны на C++

- Начиная с 90х годов, C++ – **основной язык** для низкоуровневых, высокопроизводительных систем
- Все браузеры, программы для обработки графики и видео, крупные игровые движки и т.д. написаны на C++
- Сегодня C++ *частично* замещается другими языками (Rust, Go, D)

- Начиная с 90х годов, C++ – **основной язык** для низкоуровневых, высокопроизводительных систем
- Все браузеры, программы для обработки графики и видео, крупные игровые движки и т.д. написаны на C++
- Сегодня C++ *частично* замещается другими языками (Rust, Go, D)
- Тем не менее, согласно [TIOBE Index](#), C++ всё ещё на **третьем месте** по популярности (после Python и C)

- C++ – безумно сложный язык (в стандарте сейчас около 2400 страниц)

- C++ – безумно сложный язык (в стандарте сейчас около 2400 страниц)
- Пытается совместить низкоуровневость C и высокоуровневые концепции

- C++ – безумно сложный язык (в стандарте сейчас около 2400 страниц)
- Пытается совместить низкоуровневость C и высокоуровневые концепции
- Взаимодействие всех аспектов языка приводит к тысячам нюансов и неочевидных спецэффектов

- C++ – безумно сложный язык (в стандарте сейчас около 2400 страниц)
- Пытается совместить низкоуровневость C и высокоуровневые концепции
- Взаимодействие всех аспектов языка приводит к тысячам нюансов и неочевидных спецэффектов
- В балансе между простотой использования и мощностью языковых конструкций C++ выбирает второе :)

- C++ – безумно сложный язык (в стандарте сейчас около 2400 страниц)
- Пытается совместить низкоуровневость C и высокоуровневые концепции
- Взаимодействие всех аспектов языка приводит к тысячам нюансов и неочевидных спецэффектов
- В балансе между простотой использования и мощностью языковых конструкций C++ выбирает второе :)
- Полезно изучить, даже если не планируете его использовать

- Компиляция C++ происходит так же, как компиляция C

- Компиляция C++ происходит так же, как компиляция C
- Исходный код делится на файлы, которые компилируются по отдельности, и собираются вместе линковщиком в результирующую программу/библиотеку

- Компиляция C++ происходит так же, как компиляция C
- Исходный код делится на файлы, которые компилируются по отдельности, и собираются вместе линковщиком в результирующую программу/библиотеку
- Файлы с кодом на C++ обычно имеют расширение `.cpp`, реже `.cxx` или `.cc`

- Компиляция C++ происходит так же, как компиляция C
- Исходный код делится на файлы, которые компилируются по отдельности, и собираются вместе линковщиком в результирующую программу/библиотеку
- Файлы с кодом на C++ обычно имеют расширение `.cpp`, реже `.cxx` или `.cc`
- Заголовочные файлы на C++ обычно имеют расширение `.h` или `.hpp` (чтобы отличать от C-файлов), реже `.hh` или `.hxx`

Процесс компиляции

- Компиляция C++ происходит так же, как компиляция C
- Исходный код делится на файлы, которые компилируются по отдельности, и собираются вместе линковщиком в результирующую программу/библиотеку
- Файлы с кодом на C++ обычно имеют расширение `.cpp`, реже `.cxx` или `.cc`
- Заголовочные файлы на C++ обычно имеют расширение `.h` или `.hpp` (чтобы отличать от C-файлов), реже `.hh` или `.hxx`
- **NB:** Заголовочные файлы стандартной библиотеки C++ не имеют расширения, напр. `#include <thread>`

- C++ наследует почти всё, что есть в C, но есть отличия

- C++ наследует почти всё, что есть в C, но есть отличия
- Самое важное – нет неявных приведений типов указателей:

```
int *pi = ...;
char *pc = ...;

// Можно в C, ошибка компиляции в C++
pi = pc;

// В C++ нужно явное приведение типа
pi = (int*)pc;

// Приводить к void* в C++ всё ещё можно
void *pv = pi;

// Но не обратно (ошибка компиляции в C++)
pc = pv;
```


Выделение/освобождение памяти: new/delete

- В C++ обычно память выделяют/освобождают не функциями `malloc/free`, а операторами `new/delete`:

```
// Аналог malloc(sizeof(int))
int *pi = new int;

// Аналог free(pi)
delete pi;

// Аналог malloc(N * sizeof(int))
int *array = new int[N];

// Аналог free(array)
delete [] array;
```

Выделение/освобождение памяти: new/delete

- В C++ обычно память выделяют/освобождают не функциями `malloc/free`, а операторами `new/delete`:

```
// Аналог malloc(sizeof(int))
int *pi = new int;

// Аналог free(pi)
delete pi;

// Аналог malloc(N * sizeof(int))
int *array = new int[N];

// Аналог free(array)
delete [] array;
```

- `new` и `delete` – операторы языка, они обрабатываются напрямую компилятором, и знают про тип, который выделяется

Выделение/освобождение памяти: new/delete

- В C++ обычно память выделяют/освобождают не функциями `malloc/free`, а операторами `new/delete`:

```
// Аналог malloc(sizeof(int))
int *pi = new int;

// Аналог free(pi)
delete pi;

// Аналог malloc(N * sizeof(int))
int *array = new int[N];

// Аналог free(array)
delete [] array;
```

- `new` и `delete` – операторы языка, они обрабатываются напрямую компилятором, и знают про тип, который выделяется
- Это важно для `sizeof` и для конструкторов/деструкторов (о них позже)

Выделение/освобождение памяти: new/delete

- Указатели, выделенные с помощью `malloc`, **нужно** освобождать с помощью `free`

Выделение/освобождение памяти: new/delete

- Указатели, выделенные с помощью `malloc`, **нужно** освобождать с помощью `free`
- Указатели, выделенные с помощью `new`, **нужно** освобождать с помощью `delete`

Выделение/освобождение памяти: new/delete

- Указатели, выделенные с помощью `malloc`, **нужно** освобождать с помощью `free`
- Указатели, выделенные с помощью `new`, **нужно** освобождать с помощью `delete`
- Указатели, выделенные с помощью `new[]`, **нужно** освобождать с помощью `delete[]`

Выделение/освобождение памяти: new/delete

- Указатели, выделенные с помощью `malloc`, **нужно** освобождать с помощью `free`
- Указатели, выделенные с помощью `new`, **нужно** освобождать с помощью `delete`
- Указатели, выделенные с помощью `new[]`, **нужно** освобождать с помощью `delete[]`
- Все три способа могут (и будут) внутри работать по-разному (например, запоминать количество объектов в `new[]`), и быть несовместимыми

- Указатели – один из основных инструментов в С, но бывают неудобны:

- Указатели – один из основных инструментов в С, но бывают неудобны:
 - Нужно помнить про взятие адреса `&x` и разыменовывание `*p`

- Указатели – один из основных инструментов в C, но бывают неудобны:
 - Нужно помнить про взятие адреса `&x` и разыменовывание `*p`
 - Могут быть нулевыми `int * p = NULL;`

- Указатели – один из основных инструментов в C, но бывают неудобны:
 - Нужно помнить про взятие адреса `&x` и разыменовывание `*p`
 - Могут быть нулевыми `int * p = NULL;`
 - Адрес, на который они указывают, может меняться `p = &x`

Ссылки (references)

- Указатели – один из основных инструментов в С, но бывают неудобны:
 - Нужно помнить про взятие адреса `&x` и разыменовывание `*p`
 - Могут быть нулевыми `int * p = NULL;`
 - Адрес, на который они указывают, может меняться `p = &x`
- В С++ есть альтернатива: **ссылки**

Ссылки (references)

- Тип ссылки на `T` обозначается как `T&` (например, `int&` – ссылка на `int`)

Ссылки (references)

- Тип ссылки на `T` обозначается как `T&` (например, `int&` – ссылка на `int`)

```
int x = ...;
// Здесь & - не взятие адреса, а часть типа
// int& означает ссылку на int
int &ref = x;

// Ссылку обязательно нужно проинициализировать!
int &ref2; // Ошибка компиляции

// Ссылку нельзя перенаправить на другой объект
int y = 42;
ref = y; // присваивает переменной x значение 42
// ref всё ещё указывает на x
```

Ссылки (references)

```
// Разыменование указателя возвращает ссылку
int *p = ...;
int &ref = *p;

// У ссылки можно взять адрес - получится
// адрес объекта, на который она ссылается
int *q = &ref;
assert(p == q);
```

- В целом, ссылка на объект ведёт себя как *синоним* имени этого объекта

- В целом, ссылка на объект ведёт себя как *синоним* имени этого объекта
- Самый близкий аналог в C – T `*const`, но без операторов взятия адреса и разыменования

Ссылки (references)

- В целом, ссылка на объект ведёт себя как *синоним* имени этого объекта
- Самый близкий аналог в C – T ***const**, но без операторов взятия адреса и разыменования
- В бинарном (скомпилированном) коде ссылка выглядит так же, как указатель – просто адрес в памяти

Ссылки (references)

- Ссылка может указывать на константный или неконстантный объект:

```
int x = ...;
int &ref = x;
const int &cref = x;

// Через неконстантную ссылку можно
// менять значение переменной x
ref = 42;

// Через константную ссылку нельзя -
// ошибка компиляции
cref = 42;

const int y = ...;
// Ошибка компиляции: нельзя взять неконстантную
// ссылку на константную переменную
int &yref = y;
```

Ссылки (references)

- Аргументы функции могут быть ссылками:

```
void swap(int &x, int &y)
{
    int temp = x;
    x = y;
    y = temp;
}
```

```
// Для вызова функции swap не нужны
// операторы взятия адреса:
int a = 3, b = 5;
swap(a, b);
```

Ссылки (references)

- Аргументы функции могут быть ссылками:

```
void swap(int &x, int &y)
{
    int temp = x;
    x = y;
    y = temp;
}

// Для вызова функции swap не нужны
// операторы взятия адреса:
int a = 3, b = 5;
swap(a, b);
```

- У этого есть минус: по вызову функции не всегда понятно, что копируется, а что берётся по ссылке (нужно смотреть сигнатуру вызываемой функции)

Перегрузка функций (function overloading)

- В языке С нельзя иметь несколько функций с одним именем (т.е. *перезгрузить* имя функции)

Перегрузка функций (function overloading)

- В языке C нельзя иметь несколько функций с одним именем (т.е. *перезагрузить* имя функции)
- В C++ – **можно!** Но они должны отличаться набором (количеством и/или типами) аргументов:

```
int max(int x, int y) { ... }  
  
int max(int x, int y, int z) { ... }  
  
// Ошибка компиляции: отличается только тип  
// возвращаемого значения  
double max(int x, int y) { ... }  
  
// OK - отличается тип первого аргумента  
double max(double x, int y) { ... }
```

Перегрузка функций (function overloading)

- В языке C нельзя иметь несколько функций с одним именем (т.е. *перезагрузить* имя функции)
- В C++ – **можно!** Но они должны отличаться набором (количеством и/или типами) аргументов:

```
int max(int x, int y) { ... }

int max(int x, int y, int z) { ... }

// Ошибка компиляции: отличается только тип
// возвращаемого значения
double max(int x, int y) { ... }

// OK - отличается тип первого аргумента
double max(double x, int y) { ... }
```

- Очень полезно, когда нужно реализовать похожую логику для разных типов данных или с разными настройками

- После компиляции линковщик подставляет адреса нужных функций в места их вызова

Перегрузка функций: name mangling

- После компиляции линковщик подставляет адреса нужных функций в места их вызова
- Если функция перегружена, есть несколько функций с одинаковым именем – какую он должен выбрать?

Перегрузка функций: name mangling

- После компиляции линковщик подставляет адреса нужных функций в места их вызова
- Если функция перегружена, есть несколько функций с одинаковым именем – какую он должен выбрать?
- Можно было бы научить линковщик правилам перегрузки C++, но не хотелось бы

Перегрузка функций: name mangling

- После компиляции линковщик подставляет адреса нужных функций в места их вызова
- Если функция перегружена, есть несколько функций с одинаковым именем – какую он должен выбрать?
- Можно было бы научить линковщик правилам перегрузки C++, но не хотелось бы
- Решение: **name mangling**

- При компиляции C++ имена функций преобразуются компилятором так, чтобы функции с разным количеством/типами аргументов превращались в разные *mangled* имена

Перегрузка функций: name mangling

- При компиляции C++ имена функций преобразуются компилятором так, чтобы функции с разным количеством/типами аргументов превращались в разные *mangled* имена
- Конкретный алгоритм преобразования зависит от системы и компилятора (т.е. является частью ABI)

Перегрузка функций: name mangling

- При компиляции C++ имена функций преобразуются компилятором так, чтобы функции с разным количеством/типами аргументов превращались в разные *mangled* имена
- Конкретный алгоритм преобразования зависит от системы и компилятора (т.е. является частью ABI)
- Name mangling важен для классов и неймспейсов, о них позже

Перегрузка функций: name mangling

- Под Linux можно узнать mangled имена командой
`readelf -s file.o`:

```
// -> _Z3maxii
int max(int x, int y) { ... }

// -> _Z3maxiii
int max(int x, int y, int z) { ... }

// -> _Z3maxff
double max(double x, int y) { ... }

// -> _Z3maxPcS_
char * max(char *p, char * q) { ... }
```


- Обычно говорят, что C++ – объектно-ориентированный язык

- Обычно говорят, что C++ – объектно-ориентированный язык
- Правильнее сказать, что C++ поддерживает объектно-ориентированную парадигму (методологию, стиль) программирования

ООП (объектно-ориентированное программирование)

- Обычно говорят, что C++ – объектно-ориентированный язык
- Правильнее сказать, что C++ поддерживает объектно-ориентированную парадигму (методологию, стиль) программирования
- Можно писать на C++, вообще не используя ООП-механизмы, но обычно так не делают :)

ООП (объектно-ориентированное программирование)

- Что вообще такое ООП?

ООП (объектно-ориентированное программирование)

- Что вообще такое ООП? Три принципа:

ООП (объектно-ориентированное программирование)

- Что вообще такое ООП? Три принципа:
- Инкапсуляция
 - Механизм языка, ограничивающий доступ одних компонентов программы к другим
 - Языковая конструкция, связывающая данные с методами для их обработки

ООП (объектно-ориентированное программирование)

- Что вообще такое ООП? Три принципа:
- Инкапсуляция
 - Механизм языка, ограничивающий доступ одних компонентов программы к другим
 - Языковая конструкция, связывающая данные с методами для их обработки
- Наследование
 - Механизм, позволяющий 'отнаследовать' данные и функциональность одного объекта другим, способствуя переиспользованию кода

ООП (объектно-ориентированное программирование)

- Что вообще такое ООП? Три принципа:
- Инкапсуляция
 - Механизм языка, ограничивающий доступ одних компонентов программы к другим
 - Языковая конструкция, связывающая данные с методами для их обработки
- Наследование
 - Механизм, позволяющий 'отнаследовать' данные и функциональность одного объекта другим, способствуя переиспользованию кода
- Полиморфизм
 - Способность одного и того же кода обрабатывать разные типы данных

- В C++ объектно-ориентированное программирование выражается посредством **классов**

- В C++ объектно-ориентированное программирование выражается посредством **классов**
- Проще всего смотреть на классы как на навороченные структуры (точное отличие классов и структур обсудим позднее)

- В C++ объектно-ориентированное программирование выражается посредством **классов**
- Проще всего смотреть на классы как на навороченные структуры (точное отличие классов и структур обсудим позднее)
- Объявление класса:

```
// Array - имя типа, аналогично структурам в C
class Array {
    // Поля (данные) класса, аналогично структурам в C
    int *data;
    size_t size;

    // Методы: функции, которые можно вызывать у объектов класса
    void set(size_t index, int value);
    void fill(int value);
};
```

Динамический массив без классов

```
int *array = new int[size];  
  
// Как-то работаем с массивом  
for (...)  
    array[i] = ...;  
  
delete[] array;
```

Динамический массив без классов

```
int *array = new int[size];  
  
// Как-то работаем с массивом  
for (...)  
    array[i] = ...;  
  
delete[] array;
```

- Чем плох такой динамический массив?

Динамический массив без классов

```
int *array = new int[size];  
  
// Как-то работаем с массивом  
for (...)  
    array[i] = ...;  
  
delete[] array;
```

- Чем плох такой динамический массив?
 - Нужно не забыть выделить память
 - Нужно не выйти за его границы
 - Нужно не перепутать размер (напр. в цикле **for**)
 - Нужно не забыть освободить память

Класс динамического массива

```
// array.h

class Array {
// Инкапсуляция: приватные поля недоступны снаружи объекта
private:
    size_t size;
    int *data;

// Публичный интерфейс класса
public:

    // Конструктор - создаёт объект
    Array(size_t size);

    // Деструктор - удаляет объект
    ~Array();

    // Доступ к элементам
    int get(size_t index);
    void set(size_t index, value);
};
```

Класс динамического массива

```
// array.cpp

// Реализация конструктора
Array::Array(size_t size) {
{
    // this - указатель на объект, у которого вызвали метод
    this->size = size;
    data = new int[size];
}

// Реализация деструктора
Array::~~Array() {
    delete[] data;
}

// Реализация методов
int Array::get(size_t index) {
    return data[index];
}

void Array::set(size_t index, value) {
    data[index] = value;
}
```


Класс динамического массива

```
#include <array.h>

void test_array() {
    // Вызов конструктора, size = 100
    Array a(100);

    // Ошибка компиляции: нет параметра конструктора
    Array b;

    // Ошибка компиляции: data - приватное поле
    a.data[10] = 42;

    // ОК, вызов метода класса
    a.set(10, 42);

    // Здесь автоматически(!) вызовется деструктор a
    // Как будто мы написали a.~Array() (не надо так писать самим!)
}
```

- `int x;`

- `int x;`
 - `int` – тип переменной
 - `x` – имя переменной

- `int x;`
 - `int` – тип переменной
 - `x` – имя переменной
- `Array a;`

- `int x;`
 - `int` – тип переменной
 - `x` – имя переменной
- `Array a;`
 - `Array` – класс (`class`) (он в том числе является типом!)
 - `a` – объект/экземпляр (`object/instance`) класса (во многом ведёт себя как переменная)
 - `data` – поле (`field/member`) класса
 - `set()` – метод (`method`) класса

Класс динамического массива

- Добавим *метод*, позволяющий получить размер массива:

Класс динамического массива

- Добавим *метод*, позволяющий получить размер массива:

```
class Array {  
    ...  
public:  
    ...  
  
    // Методы можно реализовывать прямо в объявлении класса!  
    // Тогда они автоматически считаются inline  
    size_t get_size() {  
        return size;  
    }  
};
```

Класс динамического массива

- Добавим *метод*, позволяющий получить размер массива:

```
class Array {  
    ...  
public:  
    ...  
  
    // Методы можно реализовывать прямо в объявлении класса!  
    // Тогда они автоматически считаются inline  
    size_t get_size() {  
        return size;  
    }  
};
```

- Пришлось назвать его `get_size()` а не `size()`, потому что такое имя конфликтовало бы с *полем* `size`

Класс динамического массива

- Добавим *метод*, позволяющий получить размер массива:

```
class Array {  
    ...  
public:  
    ...  
  
    // Методы можно реализовывать прямо в объявлении класса!  
    // Тогда они автоматически считаются inline  
    size_t get_size() {  
        return size;  
    }  
};
```

- Пришлось назвать его `get_size()` а не `size()`, потому что такое имя конфликтовало бы с *полем* `size`
- Из-за этого приватные поля часто называют с префиксами/суффиксами, например `mSize` (m от member) или `size_`

Конструкторы

- Конструктор класса – просто особенная функция (но без возвращаемого значения!), и его можно перегружать:

```
// array.h

class Array {
...
public:
    // Выделить память, оставить непроинициализированной
    Array(size_t size);

    // Выделить память, заполнить значением value
    Array(size_t size, int value);

    // Создать массив нулевого размера
    Array();
};
```

Конструкторы

```
// array.cpp

Array::Array(size_t size) {
    this->size = size;
    data = new int[size];
}

Array::Array(size_t size, int value) {
    this->size = size;
    data = new int[size];
    for (size_t i = 0; i < size; ++i)
        data[i] = value;
}

Array::Array() {
    size = 0;
    data = nullptr; // более безопасный аналог NULL
}
```

- Конструктор без аргументов `Array()` называется конструктором по умолчанию (default constructor)

Конструктор по умолчанию

- Конструктор без аргументов `Array()` называется конструктором по умолчанию (default constructor)
- Его нужно вызывать без скобок:

```
// ОК, вызов конструктора по умолчанию  
Array a;
```

Конструктор по умолчанию

- Конструктор без аргументов `Array()` называется конструктором по умолчанию (default constructor)
- Его нужно вызывать без скобок:

```
// ОК, вызов конструктора по умолчанию  
Array a;
```

- А со скобками не получится:

```
// Не ОК, это объявление функции a() без аргументов,  
// возвращающей объект класса Array  
Array a();
```

Неявный конструктор по умолчанию

- Если не написать ни одного конструктора, компилятор сам добавит конструктор по умолчанию, который ничего не делает:

```
Array::Array() {}
```

Неявный конструктор по умолчанию

- Если не написать ни одного конструктора, компилятор сам добавит конструктор по умолчанию, который ничего не делает:

```
Array::Array() {}
```

- Аналогично с деструктором:

```
Array::~~Array() {}
```


Неявный конструктор по умолчанию

- Если не написать ни одного конструктора, компилятор сам добавит конструктор по умолчанию, который ничего не делает:

```
Array::Array() {}
```

- Аналогично с деструктором:

```
Array::~~Array() {}
```

- **NB:** конструкторы/деструкторы класса всегда автоматически вызывают конструкторы/деструкторы всех полей, об этом позже

C-style инкапсуляция

```
typedef struct {  
    size_t size;  
    int *data;  
} array_t;  
  
void init(array_t *array, size_t size) {  
    array->size = size;  
    array->data = malloc(size * sizeof(int));  
}  
  
void destroy(array_t *array) {  
    free(array->data);  
}  
  
int main() {  
    array_t a;  
    init(&a, 100);  
    destroy(&a);  
}
```

- Компилятор превращает код с классами во что-то похожее на C-style инкапсуляцию

- Компилятор превращает код с классами во что-то похожее на C-style инкапсуляцию
- Все проверки на `public/private` происходят во время компиляции, в бинарном коде они уже не нужны

- Компилятор превращает код с классами во что-то похожее на C-style инкапсуляцию
- Все проверки на `public/private` происходят во время компиляции, в бинарном коде они уже не нужны
- Методы класса превращаются в функции, принимающие дополнительный параметр `this` (часто у таких функций другой calling convention, см. `thiscall`)

- Компилятор превращает код с классами во что-то похожее на C-style инкапсуляцию
- Все проверки на `public/private` происходят во время компиляции, в бинарном коде они уже не нужны
- Методы класса превращаются в функции, принимающие дополнительный параметр `this` (часто у таких функций другой calling convention, см. `thiscall`)
- Конструкторы и деструкторы, аналогично методам, превращаются в функции, и вызываются компилятором автоматически в нужных местах